

AULA 3 · ETAPA 2 · COMUNICAÇÃO E VISÃO

Quando o tópico carrega uma imagem

De mensagem de texto a fluxo de pixels — e o que muda



O que já devia estar de pé

- 1 ROS 2 Humble respondendo — `ros2 doctor` sem erro *setup · passos 3–5*
- 2 Workspace compilando com `--symlink-install` *workspace-colcon*
- 3 Exemplo da Aula 2 rodando no seu repositório *aula02-comunicacao*
- 4 Repositório `projeto-pb-<usuario>` com branch `dev` *manual-do-aluno*
- 5 `ROS_DOMAIN_ID` = seu número de chamada *no `~/.bashrc`*

Confira tudo em 5 segundos:

```
curl -sSL https://raw.githubusercontent.com/Prof-Dacio-INFNET/PBRoboticos_prof_dacio/main/recursos/check-ambiente.sh | bash
```

Faltou algo? Os primeiros 15 minutos são para isso. Depois disso, seguimos.

Hoje o tópico deixa de carregar texto

Tudo da Aula 2 continua valendo. O que muda é o tipo da mensagem — e, com ele, taxa, latência, perda de quadro e política de entrega.

É a diferença entre uma conversa e um fluxo. Numa conversa, cada mensagem importa e vale a pena repetir a que se perdeu. Num fluxo, a mensagem perdida já nasceu velha: repetir só atrasa a próxima.

Ao fim da aula você terá rodando o pipeline câmera → segmentação → contagem → serviço, que é literalmente o esqueleto dos itens 3 e 4 do TP1 — entrega em 28/08.

Como as duas horas e meia se dividem

Horário	Bloco	O que acontece
07h00–07h15	Retomada	pendências de ambiente, check-ambiente.sh, quem ainda não tem repositório
07h15–08h00	Imagem no grafo	sensor_msgs/Image, QoS de sensor, cv_bridge e o plano B
08h00–08h20	HSV e contornos	por que HSV, o problema do vermelho, morfologia, área, oclusão
08h20–09h00	Mão na massa	rodar o exemplo, trocar a cor por parâmetro, adaptar ao seu objeto
09h00–09h30	Bloco de projeto	derivações, simulação × hardware, gates do TP1, mentoria

O bloco final não é sobra de aula: é onde o seu semestre é decidido.

PARTE 1

Imagem viajando no grafo

07h15 – 08h00

- ▶ o tipo `sensor_msgs/Image`
- ▶ por que a QoS padrão está errada
- ▶ o sintoma cruel da incompatibilidade
- ▶ `cv_bridge` — e o plano B

sensor_msgs/Image não tem magia nenhuma



É a matriz do OpenCV serializada, mais os metadados de quem vai interpretá-la.

Cabeçalho

quem, quando, em que referencial

- stamp — instante da captura
- frame_id — de qual sensor

```
header.stamp  
header.frame_id
```

Geometria

como remontar a matriz

- height, width em pixels
- step = bytes de UMA linha
- is_bigendian

```
step = width * 3
```

Conteúdo

os pixels, e como lê-los

- encoding: bgr8, rgb8, mono8
- data — array contíguo de bytes

```
encoding = 'bgr8'
```

ros2 interface show sensor_msgs/msg/Image — rode agora, é mais curto do que parece.

Preencha o carimbo de tempo hoje

header.stamp parece burocracia até o TP3 — quando ele vira a única coisa que associa a imagem à pose do robô no instante da captura.

Sem stamp correto, o TF não sabe de onde o robô estava olhando quando aquele pixel foi capturado, e a projeção do que você detectou para o mapa fica errada de um jeito difícil de perceber: tudo compila, tudo roda, e o objeto aparece meio metro fora do lugar.

É barato preencher agora e caro descobrir em outubro. Custa uma linha.

Mesma lógica do frame_id: nomeie o sensor desde já, mesmo com um só.

Por que a QoS padrão está errada para câmera

RELIABLE · o padrão do ROS 2

entrega garantida, fila de 10

- retransmite o que se perdeu
- quadro atrasado ainda chega — velho
- a fila acumula e a latência cresce
- certo para comando, errado para sensor

BEST_EFFORT · perfil de sensor

KEEP_LAST, profundidade 1

- quadro perdido é quadro descartado
- sempre o mais recente, nunca uma fila
- latência constante sob carga
- é o que a sua câmera deve usar

Se um quadro se perde, retransmitir é pior do que descartar: quando chegar, já estará velho.

"O tópico existe, mas não chega nada"

- 1 **ros2 topic list mostra o tópico — ele está lá** *tudo parece certo*
- 2 **ros2 node info mostra os dois nós, publisher e subscriber** *tudo parece certo*
- 3 **ros2 topic echo fica em silêncio, para sempre** *e nenhum erro aparece*
- 4 **Causa: perfis incompatíveis — BEST_EFFORT × RELIABLE** *eles nunca se conectam*

O comando que revela os dois perfis lado a lado:

```
ros2 topic info /camera/image_raw --verbose
```

Quando o tópico existe e nada chega, suspeite de QoS antes de qualquer outra coisa. Não há mensagem de erro para isso.

cv_bridge — e o que fazer quando ele falta



A ponte entre sensor_msgs/Image e o numpy.ndarray do OpenCV, nos dois sentidos.

O caminho canônico

use este quando existir

- pacote ros-humble-cv-bridge
- imgmsg_to_cv2 e cv2_to_imgmsg
- trata encodings e conversões

```
cv2_to_imgmsg(img, 'bgr8')
```

O plano B do exemplo

módulo ponte.py, sem dependência

- tenta importar; se falhar, monta à mão
- bgr8 = 3 bytes por pixel, B-G-R
- step = largura × 3, data contíguo

```
msg.data = img.tobytes()
```

Um pacote faltando não pode travar a turma. E saber montar a mensagem à mão é o que separa quem usa a biblioteca de quem entende o que ela faz.

AO VIVO · 1

A imagem no grafo

- ▶ `ros2 launch aula03_visao visao.launch.py`
- ▶ `ros2 topic hz /camera/image_raw`
- ▶ `ros2 topic info /camera/image_raw --verbose`
- ▶ `rqt_image_view`
- ▶ `rqt_graph`



Fonte sintética por padrão: roda sem webcam, sem usbipd, sem driver.

PARTE 2

Segmentação por cor

08h00 – 08h20

- ▶ por que HSV e não BGR
- ▶ a armadilha do vermelho
- ▶ máscara, morfologia, contornos
- ▶ oclusão: o defeito que é conteúdo

Por que HSV e não BGR

BGR mistura brilho e cor

o problema

- vermelho no sol e vermelho na sombra têm valores completamente diferentes nos TRÊS canais
- "é vermelho" vira uma região torta no espaço, impossível de descrever com limiares
- qualquer mudança de luz quebra a regra

HSV separa as duas coisas

a solução

- H — que cor (matiz)
- S — quão pura (saturação)
- V — quão clara (valor)
- "é vermelho" vira uma faixa estreita em H; a variação de luz vira variação em V, que você tolera com limiar frouxo

Regra prática para começar: aperte o H, afrouxe o S e o V. Depois aperte S e V só o necessário.

No OpenCV, o vermelho mora nas duas pontas

H vai de 0 a 179 — metade do círculo de 360°, para caber em um byte. O vermelho fica perto de 0 E perto de 179.

Consequência direta: segmentar vermelho exige duas máscaras somadas (0–10 e 170–179). Azul, verde e amarelo vivem no meio do intervalo e precisam de uma só.

O sintoma de esquecer isso é traiçoeiro — a detecção funciona "às vezes", porque metade dos tons de vermelho da cena cai na faixa que você escreveu e a outra metade não. Você passa a tarde ajustando S e V achando que o problema é iluminação.

Agora você já conhece a armadilha. Quem não conhece perde, em média, uma tarde inteira com ela.

Da imagem à contagem, em cinco passos

- 1 **cvtColor — BGR para HSV** *uma linha*
- 2 **inRange — máscara binária a partir das faixas** *os seus limiares*
- 3 **morphologyEx OPEN, depois CLOSE** *tira ruído, fecha buracos*
- 4 **findContours em RETR_EXTERNAL** *só os contornos externos*
- 5 **filtro de área — descarte o que for pequeno demais** *o parâmetro que mais rende*

O filtro de área é o parâmetro mais útil e o mais subestimado:

```
ros2 param set /segmentador_hsv area_min 1200.0 # primeira coisa a ajustar quando a contagem oscila
```

OPEN remove pontos isolados; CLOSE fecha buracos dentro do objeto. Nessa ordem.

A contagem cai de 2 para 1 — e isso é conteúdo

Quando os dois círculos se sobrepõem, o pipeline conta 1. Não é defeito do exemplo: é oclusão, o problema número um de qualquer sistema de percepção real.

Reproduza em aula, registre com print e discuta no relatório do TP1: por que acontece e o que você faria a respeito — área máxima além da mínima, razão de aspecto, memória entre quadros, watershed, ou um detector por aparência no TP2.

Um parágrafo honesto sobre isso vale mais, na rubrica, do que um pipeline que finge que o problema não existe.

Regra geral da disciplina: limitação medida e explicada pontua; limitação escondida não sobrevive à arguição.

AO VIVO · 2

Trocar a cor sem tocar no código



Agora ele caça azul. Nada recompilado, nada reiniciado — é isso que o TP2 vai cobrar.

- ▶ `ros2 topic echo /vision/contagem`
- ▶ `ros2 param list /segmentador_hsv`
- ▶ `ros2 param set /segmentador_hsv h_min 100`
- ▶ `ros2 param set /segmentador_hsv h_max 130`
- ▶ `ros2 service call /vision/status \`
`std_srvs/srv/Trigger "{}"`

PARTE 3

O seu projeto, decidido hoje

09h00 – 09h30

- ▶ derivações: o mesmo núcleo, outro domínio
- ▶ simulação × hardware: a régua assimétrica
- ▶ gates: o TP como checkpoints
- ▶ mentoria individual

O núcleo técnico é quase sempre o mesmo

sensor → percepção → representação do mundo → decisão → navegação / atuação → evidência

O que muda de um projeto para outro não é a arquitetura: é o domínio. O mesmo pipeline que conta caixas em um estoque conta frutos maduros em um pomar, EPI em um canteiro de obras ou peças defeituosas em uma esteira.

Isso libera você a escolher uma aplicação que realmente te interessa sem aumentar o risco técnico. São cinco derivações para cada uma das sete famílias do catálogo, mais uma tabela de troca de setor — em recursos/derivacoes-projetos.

Escolha o domínio que você defenderia numa entrevista. É o mesmo trabalho e vale muito mais no seu portfólio.

Simulação × hardware real

Trilha S · simulada

ambiente sob controle

- desafios previsíveis, repetíveis
- sem perda de tempo com driver
- por isso: exige-se MAIS
- mais funções, mais robustez, métrica melhor

Trilha H · híbrida

recomendada

- núcleo simulado + UM elemento real
- a webcam já basta
- traz ruído, luz e latência de verdade
- sem o custo de montar um robô

Trilha R · hardware real

reconhecida no esforço

- boa parte do tempo vai para problemas que a simulação não tem
- resultado final pode ser mais modesto
- exige plano B simulado sempre vivo

Simulação não é o caminho fácil — é o caminho sem desculpas.

O TP não é uma entrega: são sete checkpoints

Gate	O que tem de estar verdadeiro	Vence
G1.0	ambiente operante — check-ambiente.sh verde	07/08
G1.1	projeto declarado: domínio, classes, trilha, plano B	11/08
G1.2	imagem no grafo, taxa medida com ros2 topic hz	15/08
G1.3	segmentação e contagem sobre o SEU objeto	20/08
G1.4	/vision/status respondendo + rqt_graph salvo	24/08
G1.5	relatório com limitação medida e discutida	26/08
G1.6	tag tp1 + entrega no Moodle	28/08

Um gate ou é verdade ou não é — e se prova com um comando. Copie o bloco para o seu PROJETO.md e faça um commit por gate.

URGENTE

O G1.1 vence em 11/08 — é o gate que trava todos os outros

Declare domínio, usuário, classes percebidas, trilha e plano B. Sem isso o G1.3 não tem o que detectar.

Quem não declara o projeto gasta duas semanas afinando HSV de um cubo vermelho genérico que não vai para lugar nenhum — e depois refaz tudo com o objeto certo, a três dias do prazo.

A leitura oficial do TP1 é na Aula 4, em 11/08. Chegue lá com o projeto já escolhido: a aula rende muito mais para quem traz uma pergunta específica do que para quem traz o projeto em branco.

Mudar de ideia depois é permitido e saudável — desde que registrado em docs/decisoes.md, com data e motivo.

O que sai daqui como tarefa

Tarefa da semana · obrigatória

vira os itens 3 e 4 do TP1

- pipeline sobre o SEU objeto, duas classes
- taxa real medida e interpretada
- /vision/status com informação do seu domínio
- parâmetros em YAML, nada fixo no código
- evidências em docs/evidencias/aula03/

Desafio 2 · opcional, sem nota

onde errar é grátis

- métrica de acerto em 20 quadros anotados
- quebrar a iluminação de propósito
- sobreviver à oclusão — e justificar como
- medir quanto a robustez custou em Hz

Prioridade quando o tempo aperta: ambiente, projeto declarado (G1.1), tarefa, e só então o desafio.

Os sete erros desta semana

Sintoma	Causa provável e cura
tópico existe, echo mudo	QoS incompatível — <code>ros2 topic info --verbose</code>
<code>ModuleNotFoundError: cv_bridge</code>	o exemplo cai no modo manual sozinho; ou instale <code>ros-humble-cv-bridge</code>
<code>rqt_image_view</code> não abre	WSLg desatualizado — <code>wsl --update</code> e reabra o terminal
webcam não abre no WSL2	faltou <code>usbipd attach</code> — ver o tutorial; enquanto isso, <code>fonte:=sintetico</code>
vermelho quase não detectado	faltou a segunda faixa de matiz, 170–179
contagem oscilando	<code>area_min</code> baixo demais, ou falta morfologia OPEN
taxa muito baixa	resolução alta demais — comece em 640×480

Antes de pedir ajuda: comando exato, saída completa, o que já tentou, e `lsb_release -a`.

PRÓXIMO ENCONTRO · 11/08

Aula 4 — leitura do TP1

terça, 07h00–09h30 · SJ205

- ▶ PROJETO.md preenchido
- ▶ pipeline rodando sobre o SEU objeto
- ▶ gates do TP1 no repositório
- ▶ uma pergunta específica